웹서비스설계　보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202412941　이름: 박주연　제출일: 10/28(수)

서비스 이름 우리의 기록

기술 조합과 버전 Vue / Node.js Express / SQLite

GitHub 저장소 URL https://github.com/qkrzoo8/jwt-board-vue-express

[서비스 소개와 기술 선택 이유]

'우리의 기록'은 사용자들이 자유롭게 게시글을 작성하고 공유할 수 있는 JWT 인증 기반 웹 게시판 서비스입니다. 로그인을 하지 않아도 게시글을 읽을 수 있지만, 글을 작성하려면 로그인이 필요하고 수정과 삭제는 본인이 작성한 글에만 가능하도록 구현했습니다. 프론트엔드는 vue, 백엔드는 Express 를 사용했습니다. 제공된 예제를 활용할 수 있고, 프론트엔드와 백엔드의 역할을 구분해서 구현하기에 적합하다고 생각해 이 조합을 선택했습니다. 데이터베이스는 SQLite 를 사용하여 서버를 종료했다가 다시 실행해도 게시글이 유지되도록 했습니다.

기존에 제공된 게시판 CRUD 코드를 바탕으로 JWT 로그인과 토큰검증, 작성자 권한 검사 기능을 추가했습니다. 또한 로그인 상태에 따라 화면이 달라지도록 수정하고, 토큰이 만료되었을 때 로그아웃 처리와 오류 메시지가 표시되도록 구현했습니다. 기본 게시판 기능을 확인한 뒤 JWT 인증과 작성자 권한 기능을 추가했습니다. Alice 와 Bob 계정을 사용하여 본인의 글은 수정·삭제할 수 있고, 다른 사람의 글은 수정·삭제할 수 없는지 확인했습니다.

교수님이 제공한 자동 검사 프로그램을 실행한 결과, 일반 API 검사 40 개와 JWT 만료를 포함한 검사 48 개를 모두 통과했습니다. 또한 Vue 프로젝트를 빌드한 후 4173 번 포트에서 실행하여 로그인과 게시글 조회가 정상적으로 작동하는 것을 확인했습니다. 서버를 재시작해도 게시글이 유지되고, 페이지를 새로고침하면 로그인 상태가 초기화되는 것도 확인했습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	frontend-vue/src/App.vue, src/api.js	비로그인 조회 가능
로그인과 JWT 발급	완료	backend-express/src/auth.js, frontend-vue/src/api.js	HS256, 기본 TTL 90 초
인증과 내 정보 조회	완료	backend-express/src/auth.js, frontend-vue/src/api.js	정상 JWT 만 접근 허용
본인 글 작성·수정·삭제	완료	backend-express/src/auth.js, frontend-vue/src/App.vue	타인의 수정 및 삭제 요청 403 처리
만료·로그아웃·오류 표시	완료	frontend-vue/src/api.js, src/App.vue	401 응답 시 로그인 상태 해제
DB 데이터 유지	완료	backend-express/data/board.sqlite	서버 재시작 후 유지 확인

웹서비스설계 02

구조와 변경 내역

[전체 서비스 구조]

사용자가 웹페이지에서 로그인하거나 게시글을 등록하면, Vue 에서 Express 서버로 API 요청을 보냅니다. Express 요청에 따라 JWT 인증과 작성자 권한을 확인하고, 게시글 데이터는 SQLite 에 저장하거나 조회합니다. 이후 처리 결과를 JSON 형태로 변환하면 Vue 에서 해당 내용을 화면에 표시하도록 구성했습니다.

[전체 요청 흐름]

브라우저(Vue) -> api.js -> Express API 서버(app.js) -> 인증 및 권한 확인(auth.js) -> SQLite 저장소(store.js) -> JSON 응답 -> Vue 화면 갱신

[로그인 요청 흐름]

사용자가 이메일과 비밀번호를 입력하면 Express 서버에서 계정 정보를 확인합니다. 로그인에 성공하면 JWT 를 발급하고, Vue 는 전달받은 토큰과 사용자 정보를 메모리에 저장합니다. 이후 인증이 필요한 요청에는 JWT 를 Authorization 헤더에 포함해 전송합니다.

[게시글 수정 요청 흐름]

사용자가 게시글을 수정 요청하면 서버에서 JWT 를 검증하여 현재 사용자를 확인합니다. 이후 게시글에 저장된 작성자 ID 와 현재 사용자 ID 를 비교합니다. 두 ID 가 같으면 수정을 허용하고, 다르면 403 FORBIDDEN 을 반환하도록 구현했습니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-express/src/auth.js — issueToken()	JWT 발급 미구현	HS256 방식의 JWT 발급 코드 작성	로그인 인증 구현
backend-express/src/auth.js — currentUserId()	사용자 ID 1 을 고정 반환	JWT 서명, 만료 시간, 사용자 정보 검증	실제 로그인 사용자 확인
backend-express/src/auth.js — assertOwner()	작성자 검사 미구현	현재 사용자와 게시글 작성자 ID 비교	다른 사용자의 글 수정·삭제 방지
frontend-vue/src/api.js — login()	로그인 API 연결 미구현	로그인 요청 및 토큰·사용자 정보 저장	Vue 로그인 기능 구현
frontend-vue/src/api.js — authorizationHeaders()	인증 헤더 미전송	요청에 Bearer 토큰 첨부	보호 API 인증
frontend-vue/src/api.js — request()	기본 API 요청과 오류 처리	401 응답 시 로그인 상태 해제	만료 및 인증 오류 처리

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
frontend-vue/src/api.js — canEdit()	모든 사용자에게 수정·삭제 허용	로그인 사용자와 작성자 ID 비교	작성자별 버튼 표시
frontend-vue/src/App.vue — save() 및 화면	기본 게시판 UI	비로그인 작성 안내 및 서비스 문구 수정	로그인 상태에 맞는 화면 제공

기존에 제공된 app.js 의 API 라우트, users.js 의 계정 검증, store.js 의 SQLite 저장 기능은 유지했습니다.

코드 작성 과정에서는 제공된 수업 코드와 AI 의 설명 및 코드 제안을 참고했습니다. 수정한 기능은 직접 서버를 실행하고 API 요청 및 브라우저 화면을 통해 동작을 확인했습니다. 구체적인 AI 활용 내역은 보고서 5 장에서 정리했습니다.

로컬 실행 기록

[실행 환경]
운영체제: macOS
Node.js: v24.21.0
npm: 11.19.0
프론트엔드: Vue, Vite 8.3.4
백엔드: Node.js Express
데이터베이스: SQLite

[백엔드 실행]
backend-express 폴더에서 npm ci 로 패키지를 설치했습니다. .env 파일에 JWT 비밀 키를 생성하여 설정한 뒤 npm start 로 서버를 실행했습니다.
서버는 http://127.0.0.1:8000 에서 실행했으며, /api/health 요청에 200 OK 와 status: ok 가 반환되는 것을 확인했습니다.

[프론트엔드 실행]
frontend-vue 폴더에서 npm ci 로 패키지를 설치했습니다. 개발 과정에서는 npm run dev 로 5173 번 포트에서 실행했으며, 구현 완료 후 npm run build 로 빌드했습니다.
빌드가 정상적으로 완료된 후 npm run preview 를 실행하여 http://127.0.0.1:4173 에서 게시판 화면이 나타나는 것을 확인했습니다.

[데이터베이스 및 실행 결과]
SQLite 파일은 backend-express/data/board.sqlite 경로에 저장되도록 구성되어 있습니다. 게시글을 작성하고 Express 서버를 종료했다가 다시 실행한 후에도 게시글의 제목, 내용, 작성자 정보가 그대로 유지되는 것을 확인했습니다.
또한 빌드된 Vue 화면에서 Alice 계정으로 로그인하고 내 정보 조회를 요청했을 때 200 OK 가 반환되는 것을 확인했습니다.
JWT 는 브라우저 메모리에만 저장하도록 구현하여 페이지를 새로고침하면 로그인 상태가 초기화되지만, SQLite 에 저장된 게시글은 유지됩니다.
검증 증거: E01(4173 번 /api/me 200), E07(Vue 빌드), E08(서버 재시작 후 게시글 조회)

상세 실행 방법: 저장소 README 및 docs/07_로컬실행안내.md

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 : backend-express/src/auth.js

함수: issueToken(user)

코드 발췌

```
export function issueToken(user) {
  return jwt.sign(
    { sub: String(user.id) },
    secret,
    {
      algorithm: "HS256",
      expiresIn: ttl,
      issuer: process.env.JWT_ISSUER || "webservice-class",
      audience: process.env.JWT_AUDIENCE || "board-api",
    }
  );
}
```

동작과 응답 코드 설명

로그인 기능을 구현하기 위해 기존에 501 NOT_IMPLEMENTED 오류를 반환하던 issueToken() 함수를 수정했습니다.

사용자가 이메일과 비밀번호를 입력하면 서버에서 기존에 제공된 계정 검증 함수를 통해 로그인 정보를 확인합니다. 올바른 계정일 경우 issueToken() 함수가 실행되며, jwt.sign()을 사용하여 JWT 를 발급하도록 구현했습니다.

JWT 에는 사용자 ID 를 문자열로 변환한 sub, 만료 시간을 나타내는 exp, 발급자 정보인 iss, 사용 대상을 나타내는 aud 가 포함됩니다. 서명 알고리즘은 HS256 을 사용했고, 토큰의 기본 유효 시간은 90 초로 설정했습니다. 또한 JWT 비밀 키는 코드에 직접 작성하지 않고 서버의 환경변수에서 불러오도록 했습니다.

로그인에 성공하면 서버는 200 OK 와 함께 accessToken, expiresIn, 사용자 정보를 반환합니다. 반대로 이메일이나 비밀번호가 틀린 경우에는 401 INVALID_CREDENTIALS, 입력값의 형식이 잘못된 경우에는 400 BAD_INPUT 을 반환하도록 구성되어 있습니다.

실제로 Alice 계정으로 로그인했을 때 JWT 가 정상적으로 발급되는 것을 확인했고, 잘못된 비밀번호를 입력했을 때는 401 INVALID_CREDENTIALS 가 반환되는 것을 확인했습니다.

3 2 브라우저 토큰 저장과 전송

파일 : frontend-vue/src/api.js

함수: login(email, password), authorizationHeaders()

코드 발췌

1. 로그인과 토큰 저장

```
export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = {
    accessToken: data.accessToken,
    user: data.user,
  };
  return data.user;
}
```

2. 토큰 전송

```
function authorizationHeaders() {
  const token = session.accessToken;
  if (!token) {
    return {};
  }
  return {
    Authorization: `Bearer ${token}`,
  };
}
```

동작과 응답 코드 설명

로그인한 사용자의 JWT 를 프론트엔드에서 사용할 수 있도록 login()과 authorizationHeaders() 함수를 수정했습니다.

먼저 login() 함수에서는 사용자가 입력한 이메일과 비밀번호를 JSON 형태로 변환하여 로그인 API 에 POST 요청을 보냅니다. 로그인에 성공하면 서버에서 전달받은 JWT 와 사용자 정보를 session 객체에 저장하고, 사용자 정보를 반환하여 화면에 로그인 상태를 표시하도록 했습니다.

JWT 는 localStorage 나 sessionStorage 가 아닌 브라우저 메모리에만 저장했습니다. 따라서 페이지를 새로고침하면 로그인 상태가 초기화되며, 실제로 새로고침 후 로그아웃 상태로 변경되는 것을 확인했습니다.

authorizationHeaders() 함수는 저장된 JWT 를 가져와 Authorization: Bearer <JWT> 형식으로 요청 헤더에 추가하는 역할을 합니다. 토큰이 없을 경우에는 빈 객체를 반환하여 잘못된 인증 헤더가 전송되지 않도록 했습니다.

이렇게 전달한 JWT 는 내 정보 조회와 게시글 등록·수정·삭제 요청에서 사용됩니다. 로그인하지 않은 사용자도 게시글 목록과 상세 내용을 조회할 수 있도록 공개 조회 기능은 유지했습니다.

로그인 정보가 잘못되어 401 INVALID_CREDENTIALS 가 반환되면 오류 메시지를 표시하고, 만료된 토큰으로 요청하여 401 TOKEN_EXPIRED 가 반환되면 저장된 로그인 상태를 해제하도록 구현했습니다. 403 FORBIDDEN 은 로그인 자체의 오류가 아니라 작성자 권한이 없는 경우이므로 자동 로그아웃하지 않도록 구분했습니다.

실제로 크롬 개발자 도구의 Network 탭에서 /api/me 요청에 Authorization 헤더가 포함된 것을 확인했습니다. 또한 정상적인 JWT 로 요청했을 때 200 OK 가 반환되는 것을 확인했습니다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 : backend-express/src/auth.js

함수: currentUserId(req)

코드 발췌

```
const authorization = req.headers.authorization;
if (!authorization || !/^Bearer [^\s]+$/i.test(authorization)) {
    throw new ApiError(401, "TOKEN_REQUIRED", "Bearer 토큰이 필요합니다.");
  }
  const token = authorization.split(" ")[1];

  let payload;

  try {
    payload = jwt.verify(token, secret, {
      algorithms: ["HS256"],
      issuer: process.env.JWT_ISSUER || "webservice-class",
      audience: process.env.JWT_AUDIENCE || "board-api",
    });
  } catch (error) {
    if (error.name === "TokenExpiredError") {
      throw new ApiError(401, "TOKEN_EXPIRED", "토큰이 만료되었습니다.");
    }
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  }
```

동작과 응답 코드 설명

JWT 를 이용해 실제 로그인한 사용자를 확인하기 위해 기존에 사용자 ID 를 1 로 고정하여 반환하던 currentUserId() 함수를 수정했습니다.

먼저 요청의 Authorization 헤더에 Bearer 토큰이 있는지 확인합니다. 토큰이 없거나 헤더 형식이 잘못된 경우에는 401 TOKEN_REQUIRED 를 반환합니다.

토큰이 전달되면 jwt.verify()를 이용해 서명과 HS256 알고리즘, 만료 시간, 발급자(iss), 사용 대상(aud)을 검증합니다. 또한 토큰에 만료 시간(exp)과 올바른 사용자 ID(sub)가 포함되어 있는지 확인하고, findPublic() 함수를 통해 실제로 존재하는 사용자인지도 검사하도록 했습니다.

검증에 성공하면 토큰에서 확인한 사용자 ID 를 반환하여 이후 게시글 처리에 사용합니다. 이처럼 검증된 사용자 ID 를 다음 처리 과정에 전달하는 부분이 요청 주체 연결점에 해당합니다.

토큰이 변조되었거나 정보가 올바르지 않으면 401 INVALID_TOKEN, 유효 시간이 지난 경우에는 401 TOKEN_EXPIRED 를 반환하도록 구분했습니다.

실제 테스트에서는 JWT 없이 내 정보를 조회했을 때 401 TOKEN_REQUIRED, 변조된 JWT 를 사용했을 때 401 INVALID_TOKEN, 만료된 JWT 를 사용했을 때 401 TOKEN_EXPIRED 가 반환되는 것을 확인했습니다. 반대로 정상적인 JWT 를 사용했을 때는 200 OK 와 함께 Alice 의 사용자 정보가 반환되었습니다.

3 4 작성자 권한 검사

파일 : backend-express/src/auth.js

함수: assertOwner(userId, authorId)

코드 발췌

```
export function assertOwner(userId, authorId) {
  if (userId !== authorId) {
    throw new ApiError(
      403,
      "FORBIDDEN",
      "본인이 작성한 게시글만 수정하거나 삭제할 수 있습니다."
    );
  }
}
```

동작과 응답 코드 설명

게시글을 작성한 사용자만 자신의 글을 수정하거나 삭제할 수 있도록 assertOwner() 함수를 구현했습니다.

userId 는 JWT 검증을 통해 확인한 현재 로그인 사용자의 ID 이고, authorId 는 데이터베이스에 저장된 게시글 작성자의 ID 입니다. 두 값을 비교하는 과정이 권한 경계 대조에 해당하며, ID 가 다르면 403 FORBIDDEN 을 반환하여 수정이나 삭제를 막도록 했습니다.

게시글을 등록할 때도 사용자가 요청 본문에 작성자 ID 를 임의로 넣는 것이 아니라, 서버에서 검증한 사용자 ID 를 작성자로 사용하도록 했습니다. 이를 통해 다른 사용자의 ID 를 이용해 게시글을 작성하거나 수정하는 문제를 방지했습니다.

프론트엔드에서도 canEdit() 함수를 이용해 본인의 글에만 수정·삭제 버튼이 보이도록 했습니다. 하지만 버튼을 숨기는 것만으로는 API 요청을 직접 보내는 것을 막을 수 없기 때문에 서버에서도 권한을 검사하도록 했습니다.

이 과정에서 JWT 가 없거나 유효하지 않으면 401, 인증에는 성공했지만 게시글 작성자가 다르면 403, 해당 게시글이 존재하지 않으면 404 를 반환하도록 구분했습니다.

실제로 Alice 가 작성한 게시글을 Bob 계정으로 수정하거나 삭제하려고 했을 때 403 FORBIDDEN 이 반환되었습니다. 이후 게시글을 다시 조회하여 제목과 내용이 변경되지 않았고, 삭제되지도 않았다는 것을 확인했습니다.

반대로 Alice 가 자신의 게시글을 수정했을 때는 200 OK, 삭제했을 때는 204 No Content 가 반환되었습니다(E09). 삭제 후 404 NOT_FOUND 는 E11 의 재조회와 E06 의 자동 검사에서 확인했습니다.

테스트 결과

실행 환경

- 운영체제 : mac OS
- Node.js : v24.21.0
- npm : 11.19.0
- 브라우저 : google Chrome
- 백엔드 : Express (http://127.0.0.1:8000)
- 프론트엔드 : Vue (http://127.0.0.1:4173)
- 데이터베이스 : SQLite

로컬 배포 확인: npm run build 성공(E07), npm run preview 후 4173 번 API 연결 확인(E01·E08).

테스트 일시 및 코드 버전: 2026.10.27 ~ 28 /

토큰 TTL: [일반 테스트 90 초 / 만료 테스트 3 초]

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인 상태에서 목록, 상세 조회	200, 정상조회	200 / 해당 없음	PASS	E08, E06
T02	Alice 계정으로 로그인	200, JWT 발급	200 / 해당 없음	P	E01, E06
T03	잘못된 비밀번호, 없는 이메일 로그인	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	P	E06
T04	토큰 없이 내 정보 조회, 글 등록	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	P	E06
T05	Alice JWT 로 글 작성 및 내 정보 조회	글 작성 201, authorId 1 / 내 정보 조회 200	201, 200 / 해당 없음 Location: /api/posts/8	P	E09b, E01
T06	Alice 가 본인 글 수정 및 삭제	200, 204	200, 204 / 해당 없음	P	E09, E11, E06
T07	Bob 이 Alice 글 수정 시도	403 FORBIDDEN	403 / FORBIDDEN	P	E02, E06
T08	Bob 이 Alice 글 삭제 시도	403 FORBIDDEN	403 / FORBIDDEN	P	E03, E06
T09	변조된 JWT 로 내 정보 및 변경 요청	401 INVALID_TOKEN	401 / INVALID_TOKEN	P	E04, E06

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T10	만료된 JWT 로 내 정보 및 변경 요청	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED	P	E05, E05b, E06
T11	로그아웃 후 내 정보 조회	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	P	직접확인
T12	Alice 가 작성자 ID 를 추가하여 글 등록	400 BAD_INPUT	400 / BAD_INPUT	P	E10(직접), E06(자동)
T13	공백 및 길이 초과 입력으로 글 작성	400 BAD_INPUT	400 / BAD_INPUT	P	직접확인, E11, E06
T14	없는 게시글 및 잘못된 ID 요청	404 NOT_FOUND / 400 BAD_INPUT	404 / NOT_FOUND, 400 / BAD_INPUT	P	E06, E11(삭제 후 조회)
T15	게시글 작성 후 서버 재시작	게시글 유지	200 / 해당 없음	P	E08
T16	로그인 후 페이지 새로고침	로그인 해제, 게시글 유지	화면에서 정상 확인	P	직접확인, E08
T17	무토큰 수정 및 삭제 및 Bob 본인 CRUD	401 / 정상 CRUD	401, 201, 200, 204	P(자동)	E06

JWT 인증 테스트

Alice 계정으로 로그인했을 때 200 OK 와 함께 JWT 가 발급되는 것을 확인했습니다. 잘못된 비밀번호를 입력했을 때는 401 INVALID_CREDENTIALS 가 반환되었고, JWT 없이 내 정보를 조회했을 때는 401 TOKEN_REQUIRED 가 반환되었습니다.
또한 JWT 의 서명 부분을 변경하여 요청했을 때는 401 INVALID_TOKEN, 유효 시간을 3 초로 설정한 뒤 만료된 토큰으로 요청했을 때는 401 TOKEN_EXPIRED 가 반환되었습니다. 브라우저에서도 토큰이 만료된 후 로그인 상태가 해제되는 것을 확인했습니다.

작성자 권한 테스트

Alice 가 작성한 게시글을 Bob 계정으로 수정하거나 삭제하려고 했을 때 403 FORBIDDEN 이 반환되었습니다. 이후 해당 게시글을 다시 조회했을 때 원래 제목과 내용이 그대로 유지되어 권한이 없는 사용자의 요청이 반영되지 않았음을 확인했습니다.
반대로 Alice 가 본인의 게시글을 수정했을 때는 200 OK, 삭제했을 때는 204 No Content 와 빈 본문이 반환되었습니다(E09). 삭제 후 재조회한 404 NOT_FOUND 는 E11 과 E06 에서 확인했습니다.
Alice 의 POST /api/posts 요청은 201 Created 와 Location: /api/posts/8 을 반환했고, 응답의 authorId 는 1 이었습니다(E09b). 같은 Alice 토큰으로 authorId:2 를 추가한 요청은 400 BAD_INPUT 으로 거부되었습니다(E10). Bob 이 authorId:1 또는 userId:1 을 추가하는 조건은 E06 의 자동 검사에서 확인했습니다.
E11 에서는 공백 제목과 공백 본문을 함께 보낸 요청이 title 입력 오류로 거부되었습니다. 이 캡처만으로 공백 본문이나 길이 초과를 각각 확인했다고 볼 수는 없습니다. E06 과 검사 코드에서 공백 제목·공백 본문·제목 101 자·본문 5001 자 입력의 400 응답을 확인했습니다. 입력 오류 후 DB 불변 여부의 개별 증거는 남아 있지 않습니다.

데이터 유지 및 화면 테스트
Express 서버를 종료했다가 다시 실행한 후에도 기존 게시글의 제목, 내용, 작성자 정보가 유지되는 것을 확인했습니다. 또한 로그인 후 페이지를 새로고침하면 로그인 상태는 초기화되지만, 게시글은 그대로 남아 있는 것을 확인했습니다.
Vue 의 프로덕션 빌드도 정상적으로 완료되었으며, 4173 번 포트에서 로그인과 내 정보 조회가 정상적으로 작동했습니다.

웹서비스설계 04 계속

HTTP 응답 확인 기록

성공 응답에는 JSON 오류 code 가 없으므로 '해당 없음'으로 표기했습니다. 개별 캡처가 없는 항목은 자동 검사 결과 또는 미확인 범위로 구분했습니다.

이번 과제에서는 API 요청이 성공하거나 실패했을 때 서버가 올바른 HTTP 상태 코드를 반환하는지 확인했습니다. 터미널에서 curl 명령어를 사용하고, Chrome 개발자 도구의 Network 탭에서 요청과 응답을 확인했습니다.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	200	200 / 해당 없음	E01, E08, E09, E06
POST /api/posts (Alice)	201 및 Location	201 Created; Location: /api/posts/8; id 8, authorId 1	E09b
DELETE /api/posts/2 (Alice)	204 및 빈 본문	204 No Content; 응답 본문 없음	E09
추가 authorId 필드	400 BAD_INPUT	400 / BAD_INPUT; "허용된 JSON 필드만 보내세요."	E10, E06
공백·길이 초과 입력	400 BAD_INPUT	E11: 공백 제목 400; 공백 본문·길이 초과는 자동 검사 통과	E11, E06
잘못된 계정	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS (자동)	E06
토큰 없음	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED; Bearer 헤더 검사 통과(자동)	E06
변조된 JWT	401 INVALID_TOKEN	401 / INVALID_TOKEN; WWW-Authenticate: Bearer	E04, E05, E06

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 서명의 만료 JWT	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED; WWW-Authenticate: Bearer	E05, E06
다른 작성자의 변경	403 FORBIDDEN	403 / FORBIDDEN; 원문 재조회 200	E02, E03, E06
없는 글	404 NOT_FOUND	404 / NOT_FOUND; 삭제 후 /api/posts/2 조회	E11, E06
헤더 형식·개별 클레임 오류	401 (명세·코드)	개별 실행 증거 미확인	auth.js 구현 확인만 함

보호 API 의 인증 응답 헤더

E05 의 401 응답에서 WWW-Authenticate: Bearer 헤더를 확인했습니다. 무토큰 요청의 401 TOKEN_REQUIRED 및 Bearer 헤더 검사는 E06 과 자동 검사 코드에서 확인했습니다.

JWT 의 서명을 변조했을 때는 401 INVALID_TOKEN, 유효 시간이 지난 토큰으로 요청했을 때는 401 TOKEN_EXPIRED 가 반환되었습니다. 두 경우 모두 서버에서 인증 요청을 거부하는 것을 확인했습니다.

401 과 403 의 화면 처리

401 은 정상적으로 인증되지 않은 경우이고, 403 은 인증에는 성공했지만 해당 작업을 수행할 권한이 없는 경우입니다.

Vue 에서는 401 응답을 받으면 저장된 토큰과 사용자 정보를 삭제하도록 구현했습니다. 실제로 만료된 JWT 로 내 정보를 조회했을 때 오류 메시지가 나타나고 로그아웃 상태로 변경되는 것을 확인했습니다.

반면 403 은 토큰 자체의 오류가 아니므로 로그인 상태를 유지하도록 구성했습니다. Bob 으로 로그인한 경우 Alice 의 게시글에서 수정·삭제 버튼이 표시되지 않는 것도 확인했습니다.

실패한 수정·삭제 후 데이터 유지

Bob 의 JWT 로 Alice 가 작성한 게시글을 수정하거나 삭제하려고 했을 때 403 FORBIDDEN 이 반환되었습니다.

이후 GET /api/posts/2 로 다시 조회한 결과, 게시글의 제목과 내용이 변경되지 않았으며 작성자와 수정 시간도 그대로 유지되는 것을 확인했습니다. 삭제 요청 이후에도 게시글이 정상적으로 조회되어 권한이 없는 사용자의 요청이 데이터베이스에 반영되지 않았음을 확인했습니다.

토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과

토큰 없이 존재하지 않는 게시글에 수정 요청을 보냈을 때는 401 TOKEN_REQUIRED 가 반환되었습니다. 정상적인 JWT 를 포함하여 존재하지 않는 게시글에 수정·삭제 요청을 보냈을 때는 404 NOT_FOUND 가 반환되었습니다. 이를 통해 서버가 JWT 인증을 먼저 수행하고, 인증에 성공한 후 게시글의 존재 여부를 확인하는 것을 확인했습니다. 무토큰 요청 결과는 직접 확인했으며 별도 캡처는 첨부하지 않았습니다. 정상 토큰을 사용한 수정·삭제 검사는 E06 의 자동 검사 결과와 연결했습니다.

테스트 증거

로그인과 토큰 전송

Alice 계정으로 로그인한 후 내 정보 조회 기능을 실행했습니다. Chrome 개발자 도구의 Network 탭에서 /api/me 요청을 확인한 결과, Authorization 헤더에 Bearer 토큰이 포함되어 있었으며 서버에서 200 OK 가 반환되었습니다. 이를 통해 Vue 에서 저장한 JWT 가 Express 서버로 정상적으로 전달되는 것을 확인했습니다.

E01_로그인_JWT 전송.png
E01b_Bearer 헤더_토큰가림.png

다른 작성자의 수정과 삭제

Alice 가 작성한 게시글을 Bob 계정으로 수정하거나 삭제하는 요청을 직접 보냈습니다. 두 요청 모두 서버에서 403 FORBIDDEN 이 반환되어 권한이 없는 사용자의 접근이 차단되는 것을 확인했습니다.

이후 게시글을 다시 조회했을 때 기존 제목과 내용이 그대로 유지되어 수정이나 삭제가 이루어지지 않았음을 확인했습니다.

E02_작성자_수정거부.png
E03_작성자_삭제거부.png

토큰 변조와 만료

정상적으로 발급받은 JWT 의 서명 부분을 일부 변경한 뒤 내 정보 조회를 요청했습니다. 그 결과 서버에서 401 INVALID_TOKEN 이 반환되었습니다.
또한 JWT 유효 시간을 3 초로 변경하고 5 초 후 다시 요청했을 때 401 TOKEN_EXPIRED 가 반환되었습니다. Vue 화면에서도 토큰 만료 메시지가 표시되고 로그인 상태가 해제되는 것을 확인했습니다.

E04_JWT 변조.png
E05_JWT 만료.png

자동 검사와 로컬 배포 증거

교수님이 제공한 check_api.py 를 이용해 API 검사를 진행했습니다. 일반 최종 검사에서는 40 개 항목을 모두 통과했고, JWT 만료를 포함한 검사에서는 48 개 항목을 모두 통과했습니다.

E05b 는 5173 번 포트에서 확인한 개발 화면입니다. 최종 빌드 후 4173 번 포트에서 로그인과 API 연결을 확인한 결과는 E01 과 E08 에 정리했습니다. Express 서버를 재시작한 후에도 기존 게시글이 유지되는 것도 확인했습니다.

E05b_만료후_자동로그아웃_UI.png

API 자동 검사 결과

교수님이 제공한 tools/check_api.py 로 일반 최종 검사 40/40, 만료 포함 검사 48/48 항목을 통과했습니다. 실행 명령은 python3 tools/check_api.py --mode final 및 --mode final --expiry 입니다. 자동 검사는 UI 와 서버 재시작 검사를 포함하지 않습니다. T03·T04·T07~T10·T12~T14·T17 의 자동 검사 범위는 테스트 표에 구분했습니다.

E06_자동검사_48PASS.png

Vue 빌드와 데이터 유지

Vue 프로젝트를 npm run build 로 빌드한 후 npm run preview 로 4173 번 포트에서 실행했습니다. E07 은 빌드 성공 로그이고, E08 은 서버 재시작 후 기존 게시글 조회 및 목록·상세 API 200 응답 화면입니다(T01·T15). 새로고침 후 로그인 해제와 게시글 유지를 직접 확인했으며, E08 을 참고 자료로 첨부했습니다.
E07_프로덕션빌드.png
E08_데이터유지.png

글 수정과 삭제/ 작성 location/ 작성자 ID 위조 거부/ 입력 오류와 삭제 후 조회

Alice 의 PUT /api/posts/2 는 200 OK 와 수정된 게시글 JSON 을 반환했습니다. 이어서 DELETE /api/posts/2 는 204 No Content 와 빈 본문을 반환했습니다(T06). 삭제 후 재조회 404 는 E11 과 E06 에서 확인했습니다.
Alice 의 POST /api/posts 는 201 Created 와 Location: /api/posts/8 을 반환했습니다(T05). 응답 JSON 의 id 는 8, authorId 는 1, authorName 은 Alice 입니다. 내 정보 조회 200 응답은 E01 과 연결했습니다.
Alice 토큰으로 authorId:2 를 추가한 POST /api/posts 요청이 400 BAD_INPUT 으로 거부되었으며, 메시지는 "허용된 JSON 필드만 보내세요."입니다(T12). Bob 이 authorId:1 또는 userId:1 을 추가한 조건은 E06 의 자동 검사에서 별도로 통과했습니다.
삭제 후 GET /api/posts/2 는 404 NOT_FOUND 였습니다. 공백 title·content 를 함께 보낸 작성 요청은 400 BAD_INPUT 과 "title 입력을 확인하세요."를 반환했습니다(T13). 공백 본문·길이 초과 개별 응답은 이 사진으로 확인한 것으로 표기하지 않고 E06 의 자동 검사 결과와 구분했습니다.

E08_데이터유지.png
E09_본인글_수정 200_삭제 204.png
E09b_게시글작성 201_Location.png
E10_작성자위조_400_BAD_INPUT.png
E11_입력검사_400_BAD_INPUT.png

웹서비스설계 05

AI 활용과 참고 자료

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
OpenAI ChatGPT	JWT 발급 및 검증 구현	auth.js	제안 코드 참고 및 수정
OpenAI ChatGPT	작성자 권한 검사 구현	auth.js	제안 코드 적용 및 테스트
OpenAI ChatGPT	로그인 연결 및 오류 처리	auth.js, App.vue	제안 코드 적용 및 일부 화면 수정
OpenAI ChatGPT	테스트 방법 및 보고서 작성	테스트 명령어, 보고서	설명 참고 및 결과 정리

대표 사례

[사례 1] JWT 발급 및 검증 기능 구현

도구 / 날짜: ChatGPT / 2026.10.27

요청한 내용: Express 에서 JWT 를 발급하고 검증하는 방법과 기존 코드의 수정 위치를 질문했습니다.

AI 가 제안한 내용: jsonwebtoken 라이브러리를 이용한 HS256 토큰 발급, JWT 비밀 키 설정, 서명 및 만료 시간 검증 방법을 안내받았습니다.

실제 반영 위치: backend-express/src/auth.js 의 issueToken(), currentUserId()

내가 확인하고 수정한 내용: 처음에는 AI 가 auth.js 전체 코드를 교체하도록 안내했지만, 기존에 제공된 코드까지 모두 삭제해도 되는지 의문이 들어 다시 질문했습니다. 이후 기존 코드 구조는 유지하고 JWT 발급과 검증에 필요한 함수만 수정했습니다. 또한 JWT 비밀 키는 코드에 직접 작성하지 않고 .env 파일에 설정했습니다.

검증 방법과 결과: T02, T03, T04, T09, T10 과 관련된 요청을 직접 테스트했습니다. 기존에는 로그인 시 501 NOT_IMPLEMENTED 가 반환되도록 되어 있었지만, 수정 후에는 정상 로그인 시 200 OK 와 JWT 가 반환되었습니다. 잘못된 비밀번호는 401 INVALID_CREDENTIALS, 토큰 누락은 401 TOKEN_REQUIRED, 변조된 토큰은 401 INVALID_TOKEN, 만료된 토큰은 401 TOKEN_EXPIRED 가 반환되는 것을 확인했습니다.

[사례 2] 게시글 작성자 권한 검사 구현

도구 / 날짜: ChatGPT (GPT-6) / 2026.10.27

요청한 내용: 로그인한 사용자만 자신의 게시글을 수정하거나 삭제할 수 있도록 작성자 권한을 검사하는 방법을 질문했습니다.

AI 가 제안한 내용: JWT 에서 확인한 사용자 ID 와 게시글에 저장된 작성자 ID 를 비교하고, 두 값이 다르면 403 FORBIDDEN 을 반환하는 코드를 제안받았습니다.

실제 반영 위치: backend-express/src/auth.js 의 assertOwner()

내가 확인하고 수정한 내용: 기존 assertOwner() 함수에는 작성자 권한을 검사하는 코드가 없었기 때문에 AI 가 제안한 조건문을 해당 함수에 추가했습니다. 기존 게시판 CRUD 코드는 유지하고 작성자 권한을 확인하는 부분만 수정했습니다. 또한 프론트엔드에서 버튼을 숨기는 것과 서버에서 권한을 검사하는 것이 별개라는 점을 확인했습니다.

검증 방법과 결과: T06, T07, T08 과 관련된 테스트를 진행했습니다. Alice 가 작성한 게시글을 Bob 계정으로 수정하거나 삭제하려고 했을 때 모두 403 FORBIDDEN 이 반환되었습니다. 이후 게시글을 다시 조회하여 제목과 내용이 변경되지 않았고 삭제되지도 않았음을 확인했습니다. 반대로 Alice 가 자신의 게시글을 수정했을 때는 200 OK, 삭제했을 때는 204 No Content 가 반환되었습니다.

[사례 3] Vue 로그인 연결 및 오류 처리

도구 / 날짜: ChatGPT (GPT-6) / 2026.10.27~10.28

요청한 내용: Vue 에서 로그인 API 를 연결하고 JWT 를 서버에 전송하는 방법, 로그인 상태에 따라 수정·삭제 버튼을 표시하고 오류를 처리하는 방법을 질문했습니다.

AI 가 제안한 내용: 로그인 성공 시 JWT 를 메모리에 저장하고, API 요청에 Authorization 헤더를 추가하는 방법을 안내받았습니다. 또한 401 응답 시 로그인 상태를 해제하고, 게시글 작성자에 따라 수정·삭제 버튼을 표시하는 코드를 제안받았습니다.

실제 반영 위치: frontend-vue/src/api.js 의 login(), authorizationHeaders(), request(), canEdit() 및 frontend-vue/src/App.vue 의 save()와 화면 안내 문구

내가 확인하고 수정한 내용: AI 가 제안한 코드를 기존 함수에 적용하고, 실제 화면에서 동작하는지 확인했습니다. 비로그인 상태에서 글을 작성하면 처음에는 'Bearer 토큰이 필요합니다.'라는 오류가 표시되었지만, 사용자에게 더 이해하기 쉽도록 '게시글을 작성하려면 먼저 로그인해 주세요.'라는 안내 문구로 변경했습니다. 또한 인증 구현을 완료한 후 기존의 '인증 미완성' 안내 문구도 실제 서비스 설명으로 수정했습니다.

검증 방법과 결과: T05, T10, T11, T16 과 관련된 기능을 브라우저에서 확인했습니다. Alice 로 로그인한 후 /api/me 요청에 JWT 가 포함되고 200 OK 가 반환되는 것을 확인했습니다. 토큰이 만료되었을 때는 401 TOKEN_EXPIRED 가 반환되고 로그인 상태가 해제되었습니다. 또한 Bob 으로 로그인했을 때 Alice 게시글의 수정·삭제 버튼이 표시되지 않았으며, 페이지를 새로고침하면 로그인 상태가 초기화되는 것도 확인했습니다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
교수자 제공 과제 공지 및 Vue·Express 시작 코드	auth.js, api.js, App.vue	기존 CRUD 구조를 유지하고 인증·권한 기능 구현
docs/02_API 명세.md 및 03_구현체크리스트.md	API·JWT 구현	HTTP 상태, 오류 code, JWT·작성자 검사 요구사항
docs/05_테스트시나리오.md, tools/check_api.py	T01~T17, HTTP 증거표	시나리오와 자동 검사 범위 및 결과 대조
docs/07_로컬실행안내.md	설치·빌드·로컬 실행	백엔드 실행과 Vue 빌드·4173 번 확인 절차

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

사용자가 요청 본문의 authorId 를 직접 지정할 수 있다면 다른 사용자의 ID 를 넣어 게시글을 작성하는 문제가 발생할 수 있습니다.

이를 방지하기 위해 게시글을 작성할 때는 사용자가 전달한 작성자 ID 를 사용하지 않고, 서버에서 JWT 를 검증하여 확인한 사용자 ID 를 작성자로 설정하도록 했습니다. 또한 요청 본문에 authorId 와 같은 추가 필드를 포함하면 400 BAD_INPUT 을 반환하도록 구성되어 있습니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

화면에서 수정 버튼을 숨기더라도 사용자가 curl 등의 도구를 이용해 API 요청을 직접 보낼 수 있기 때문에 버튼을 숨기는 것만으로는 충분하지 않습니다.

이를 해결하기 위해 프론트엔드에서는 canEdit() 함수로 작성자에 따라 버튼 표시를 제한하고, 백엔드에서는 assertOwner() 함수로 실제 수정·삭제 권한을 검사하도록 했습니다.

서버에서는 JWT 로 확인한 사용자 ID 와 게시글 작성자 ID 가 다르면 403 FORBIDDEN 을 반환합니다.
실제로 Bob 이 Alice 의 게시글을 수정하거나 삭제하려고 했을 때 요청이 거부되는 것을 확인했습니다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 JWT 에 들어 있는 정보를 읽는 기능이고, verify 는 토큰의 서명과 유효성을 검사하는 기능입니다.
따라서 decode 만 사용하면 토큰이 변조되었는지 확인할 수 없습니다.

이번 과제에서는 jwt.verify()를 사용하여 HS256 알고리즘과 서명, 만료 시간, 발급자(iss), 사용 대상(aud)을 검증했습니다. 추가로 사용자 ID(sub)와 만료 시간(exp)의 형식을 확인하고, 실제 존재하는 사용자 ID 인지도 검사했습니다.

테스트에서는 토큰을 변조했을 때 401 INVALID_TOKEN, 만료된 토큰을 사용했을 때 401 TOKEN_EXPIRED 가 반환되는 것을 확인했습니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 브라우저 메모리에 저장된 JWT 와 사용자 정보를 삭제하여 로그인 상태를 해제하는 기능입니다.

토큰 만료는 JWT 에 설정된 유효 시간이 지나 서버에서 해당 토큰을 더 이상 인증에 사용하지 못하게 되는 것입니다. 이번 과제에서는 만료된 JWT 로 요청하면 401 TOKEN_EXPIRED 를 반환하고, Vue 에서도 로그인 상태를 해제하도록 했습니다.

새로고침을 하면 브라우저 메모리에 저장된 로그인 정보가 초기화되므로 로그아웃 상태로 돌아갑니다.
하지만 SQLite 에 저장된 게시글 데이터는 그대로 유지됩니다.

또한 로그아웃하더라도 이미 복사된 JWT 가 서버에서 즉시 무효화되는 것은 아닙니다. 해당 토큰은 유효 시간이 지나기 전까지 다시 사용될 수 있으며, 이번 과제에서는 별도의 토큰 폐기 기능을 구현하지 않았습니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

과제를 진행하면서 헷갈렸던 부분은 Chrome 개발자 도구에서 내 정보 조회 API 의 응답이 200 OK 가 아닌 304 Not Modified 로 표시되었던 상황입니다.

처음에는 JWT 전송이나 API 구현에 문제가 있는 것으로 생각했지만, 확인해 보니 브라우저가 이전 응답을 캐시에 저장해 두었기 때문에 나타나는 결과였습니다.

이를 확인하기 위해 Network 탭에서 Disable cache 를 설정하고 페이지를 새로고침한 뒤 Alice 계정으로 다시 로그인했습니다. 이후 내 정보 조회를 요청했을 때 200 OK 가 반환되었고, Authorization 헤더에 JWT 도 정상적으로 포함된 것을 확인했습니다.

이 과정에서는 소스 코드를 수정하지 않았으며, 브라우저 캐시 설정을 변경하여 다시 검증했습니다. 이를 통해 HTTP 응답 코드가 예상과 다를 때는 코드뿐만 아니라 브라우저의 요청 환경도 확인해야 한다는 것을 알게 되었습니다.

현재 남은 문제와 다음에 개선할 점: [무토큰으로 없는 글을 수정하는 복합 조건, 로그아웃·새로고침 전후의 요청과 화면을 추가 기록할 계획입니다. 별도의 토큰 폐기 기능은 구현하지 않았습니다.]

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거
GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출